Mini Project 3

1. TITLE - Cloud-Native Airline Reservation System using AWS ECS Fargate, ECR & ALB

"Sky Route – Cloud-Native Airline Reservation Application using AWS ECS Fargate, ECR and Application Load Balancer"

2. Project Objective-

The objective of this project is to design, containerize, and deploy a **cloud-native airline reservation web application** using **Docker** and **AWS managed container services**.
The application is deployed using **ECS Fargate**, container images are stored in **Amazon ECR**, and traffic is distributed using an **Application Load Balancer (ALB)**.

3. Application Overview

The application is a **Node.js / Web-based Airline Ticketing System** that allows users to:

- View available flights
- Simulate ticket booking
- Access the application via browser

The app is packaged as a **Docker container** and deployed on AWS ECS.

4. Docker Image Creation

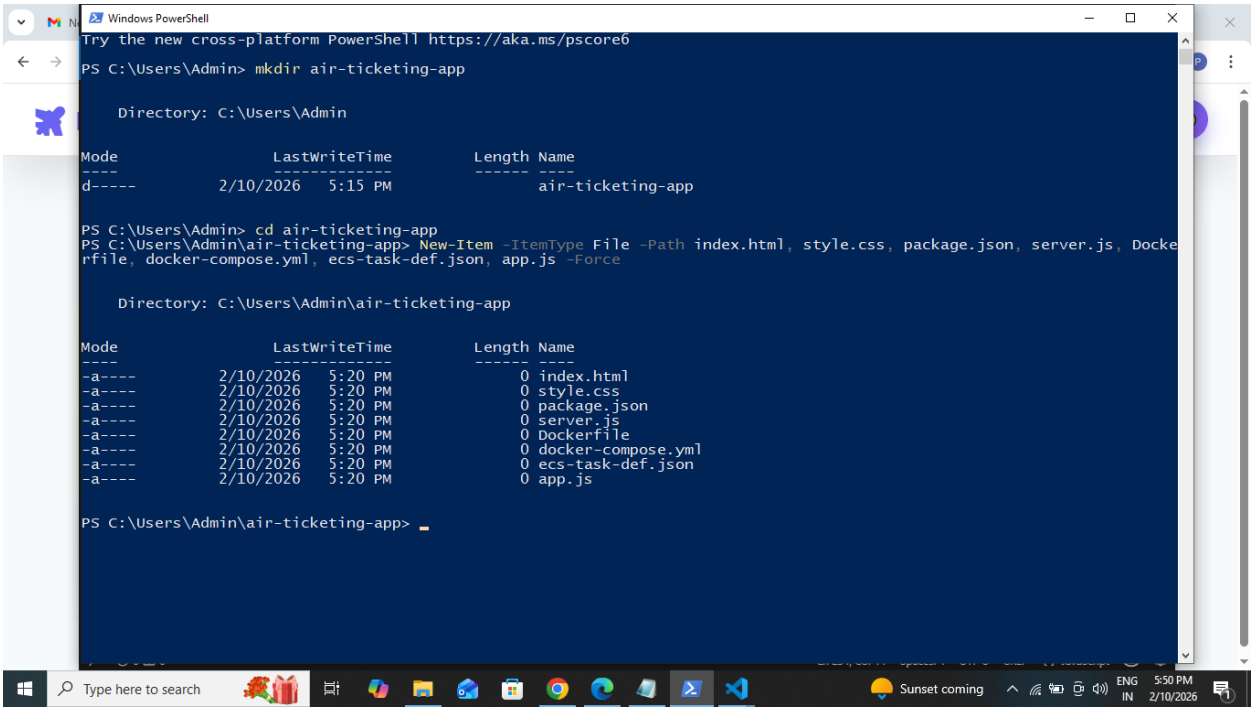
Why Docker?

Docker ensures:

- Application portability
- Consistent runtime environment
- Easy deployment to cloud platforms

Explanation:
A Dockerfile was created to define how the application image is built.
It includes:

a) Base image (Node.js) b)Application dependencies c) Port exposure d) Startup command.



The following commands are used—

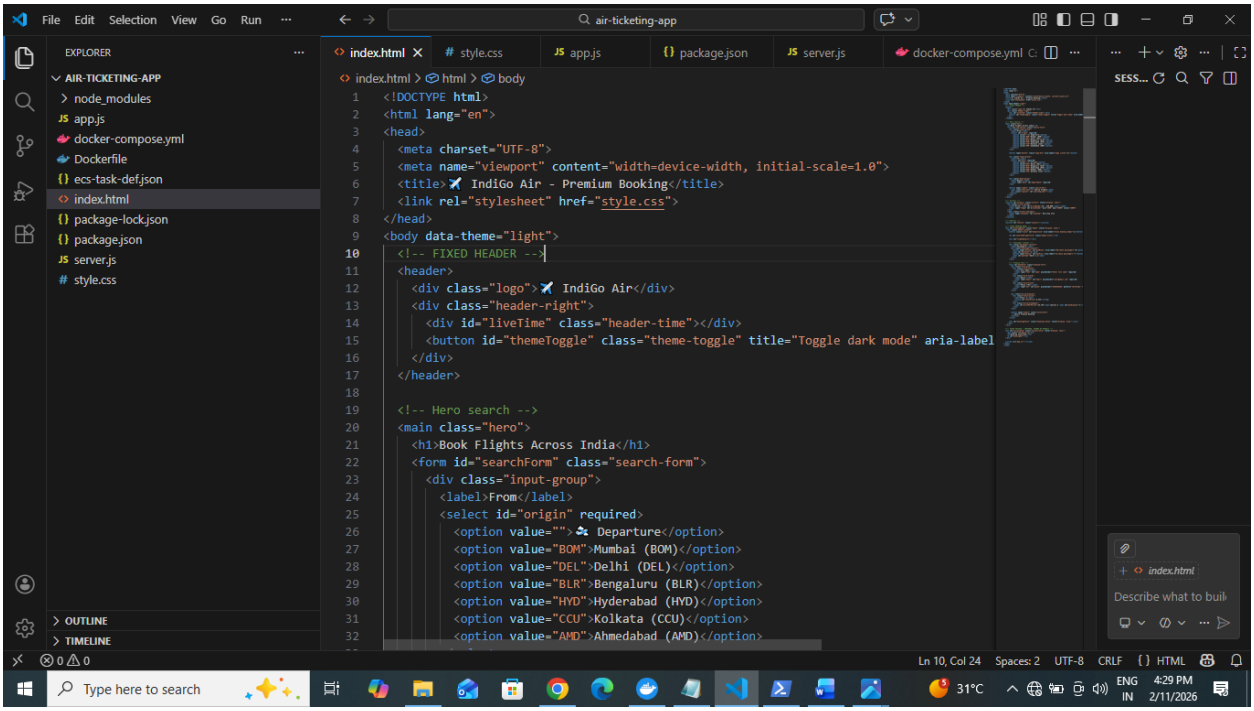
mkdir air-ticketing-app # Create project folder

cd air-ticketing-app # Go inside the project folder

New-Item -ItemType File -Path index.html, style.css, package.json, server.js, Dockerfile, docker-compose.yml, ecs-task-def.json, app.js -Force # Create all files

Get-Content package.json # Show package.json content

5. create app in visual studio—



Create HTML,CSS,APP.JS, DOCKER COMPOSE,DOCKER FILE,APP.JS files in vs....

Then using following commands

npm install # Install Node.js dependencies

npm --version # Check npm version

login --username AWS --password-stdin 739275449784.dkr.ecr.us-east-1.amazonaws.com # Login to ECR

docker build -t indigo-air . # Build Docker image

node --version # Check Node.js version

node server.js # Start Express server on port 3000

docker --version # Check Docker version

docker compose up --build # Build and start containers

docker compose down # Stop and remove containers

aws configure # Configure AWS CLI credentials and region

6.Pushing Docker Image to ECR---

```
Windows PowerShell
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:us-east-1:733996454318:repository/indigo-air",
        "registryId": "733996454318",
        "repositoryName": "indigo-air",
        "repositoryUri": "733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air",
        "createdAt": "2026-02-10T22:29:22.398000+05:30",
        "imageTagMutability": "MUTABLE",
        "imageScanningConfiguration": {
            "scanOnPush": false
        },
        "encryptionConfiguration": {
            "encryptionType": "AES256"
        }
    }
}

PS C:\Users\Admin>
PS C:\Users\Admin> aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 733996454318.dkr.ecr.u
s-east-1.amazonaws.com
Login Succeeded
PS C:\Users\Admin>
PS C:\Users\Admin> docker tag aws.com/indigo-air:latest 733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest
Error response from daemon: No such image: aws.com/indigo-air:latest
PS C:\Users\Admin> docker tag aws.com/indigo-air:latest 733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest
Error response from daemon: No such image: aws.com/indigo-air:latest
PS C:\Users\Admin> docker images
                                                                                                    i Info →   U  In UseIMAGE
                                                   ID             DISK USAGE   CONTENT SIZE   EXTRA
733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest   ec70c0a0f68c        208MB           50MB
air-ticketing-app-indigo-app:latest                              15b94017fc1e        208MB           50MB
indigo-air:latest                                                ec70c0a0f68c        208MB           50MB
PS C:\Users\Admin> docker push 733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest
The push refers to repository [733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air]
bfe747a3973b: Pushed
e31b20165522: Pushed
589002ba0eae: Pushed
5b279e7b04d1: Pushed
15e2802d0658: Pushed
60c25d7c4895: Pushed
db185ce96b82: Pushed
50888cc464df: Pushed
ad6d96c196e3: Pushed
eb87f4721c91: Pushed
```

```
Windows PowerShell
PS C:\Users\Admin> docker tag aws.com/indigo-air:latest 733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest
Error response from daemon: No such image: aws.com/indigo-air:latest
PS C:\Users\Admin> docker images
                                                                                                    i Info →   U  In UseIMAGE
                                                   ID             DISK USAGE   CONTENT SIZE   EXTRA
733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest   ec70c0a0f68c        208MB           50MB
air-ticketing-app-indigo-app:latest                              15b94017fc1e        208MB           50MB
indigo-air:latest                                                ec70c0a0f68c        208MB           50MB
PS C:\Users\Admin> docker push 733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest
The push refers to repository [733996454318.dkr.ecr.us-east-1.amazonaws.com/indigo-air]
bfe747a3973b: Pushed
e31b20165522: Pushed
589002ba0eae: Pushed
5b279e7b04d1: Pushed
15e2802d0658: Pushed
60c25d7c4895: Pushed
db185ce96b82: Pushed
50888cc464df: Pushed
ad6d96c196e3: Pushed
eb87f4721c91: Pushed
06a1039c511c: Pushed
latest: digest: sha256:ec70c0a0f68cc783dd656df14394e9c8f33a78b2666a11aefc15ebc7ee6e5996 size: 856
PS C:\Users\Admin> aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 733996454318.dkr.ecr.
ap-south-1.amazonaws.com
Login Succeeded
PS C:\Users\Admin>
PS C:\Users\Admin> docker tag indigo-air:latest 733996454318.dkr.ecr.ap-south-1.amazonaws.com/indigo-air:latest
PS C:\Users\Admin> docker push 733996454318.dkr.ecr.ap-south-1.amazonaws.com/indigo-air:latest
The push refers to repository [733996454318.dkr.ecr.ap-south-1.amazonaws.com/indigo-air]
bfe747a3973b: Pushed
e31b20165522: Pushed
5b279e7b04d1: Pushed
589002ba0eae: Pushed
15e2802d0658: Pushed
50888cc464df: Pushed
60c25d7c4895: Pushed
06a1039c511c: Pushed
db185ce96b82: Pushed
ad6d96c196e3: Pushed
eb87f4721c91: Pushed
latest: digest: sha256:ec70c0a0f68cc783dd656df14394e9c8f33a78b2666a11aefc15ebc7ee6e5996 size: 856
PS C:\Users\Admin>
```

Using following commands to ---

aws ecr create-repository --repository-name indigo-air --region us-east-1 # Create ECR repository

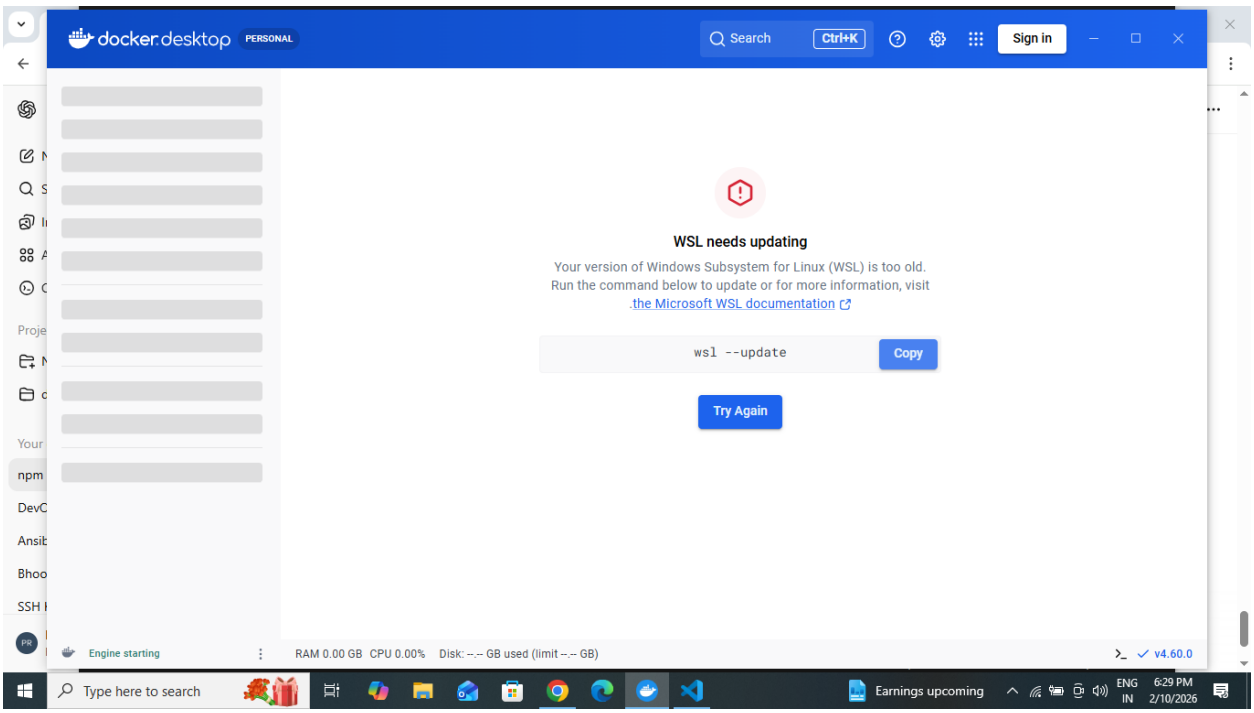
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 739275449784.dkr.ecr.us-east-1.amazonaws.com # Login to ECR

docker build -t indigo-air . # Build Docker image

docker tag indigo-air:latest 739275449784.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest # Tag image for ECR

docker push 739275449784.dkr.ecr.us-east-1.amazonaws.com/indigo-air:latest # Push image to ECR

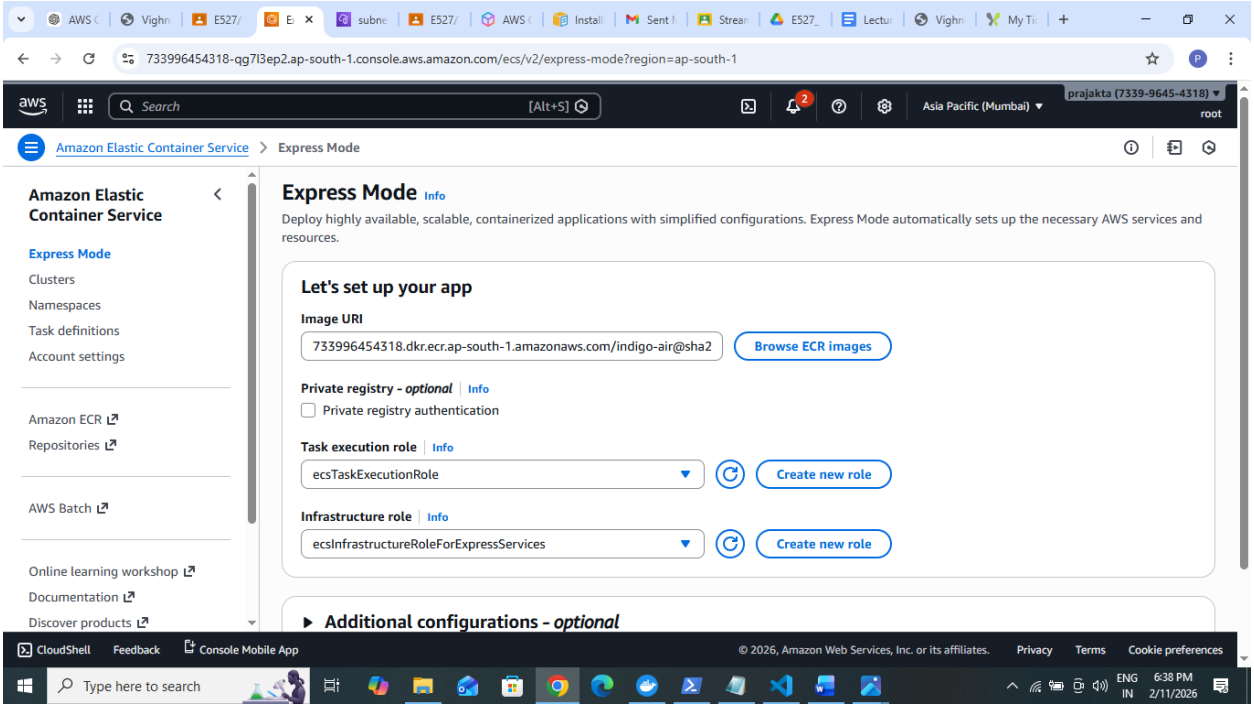
7. open docker tool-

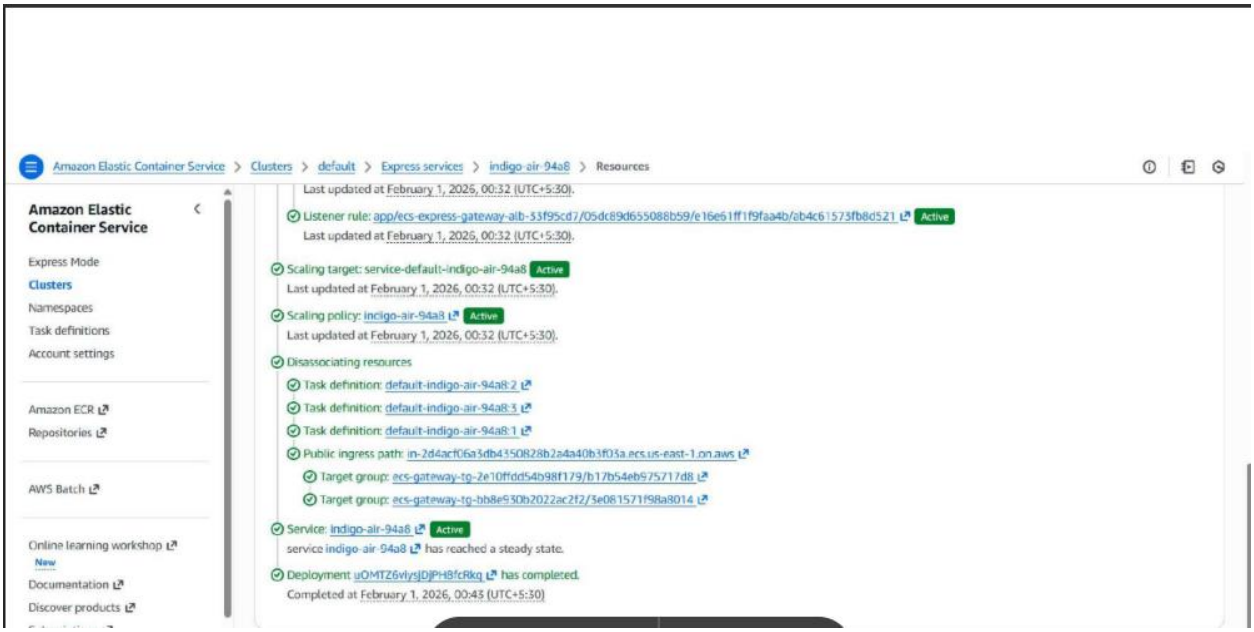


7. ECS Cluster Creation

Why ECS Cluster?

An ECS cluster acts as a **logical grouping** to run containerized applications.



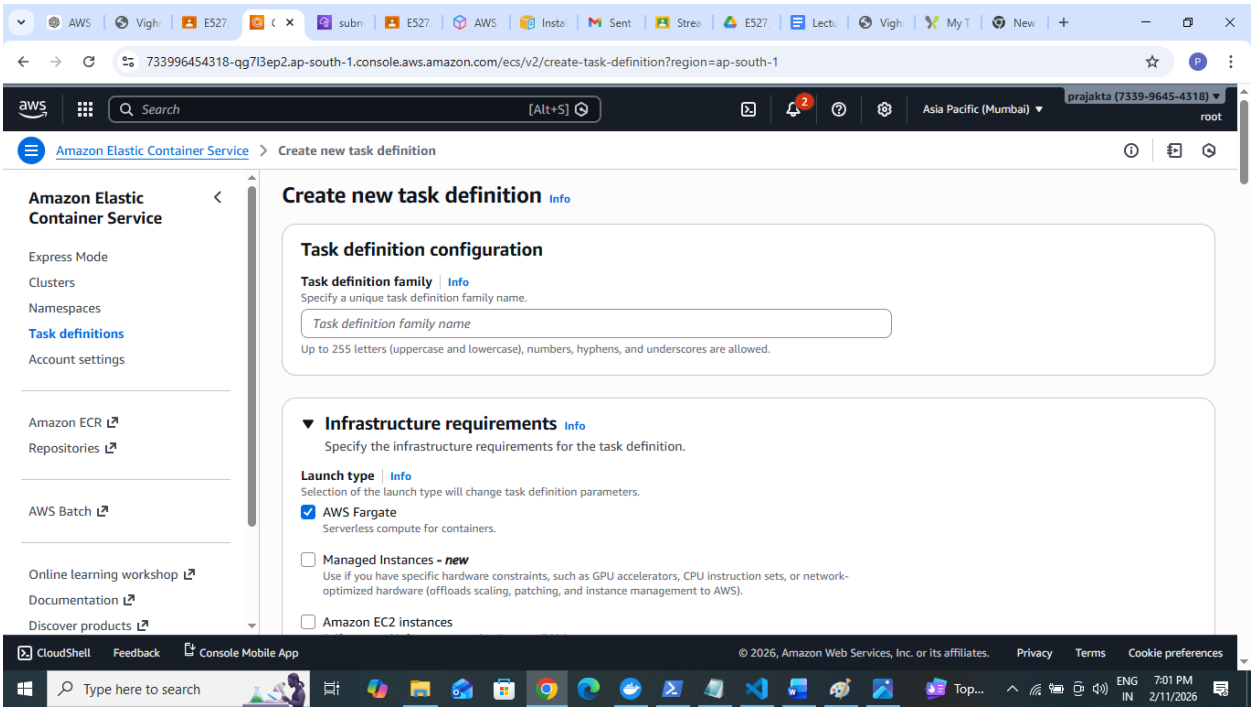


8. Task Definition Creation

Why Task Definition?

A task definition defines:

- Which Docker image to run
- CPU & memory requirements
- Container ports
- Logging configuration

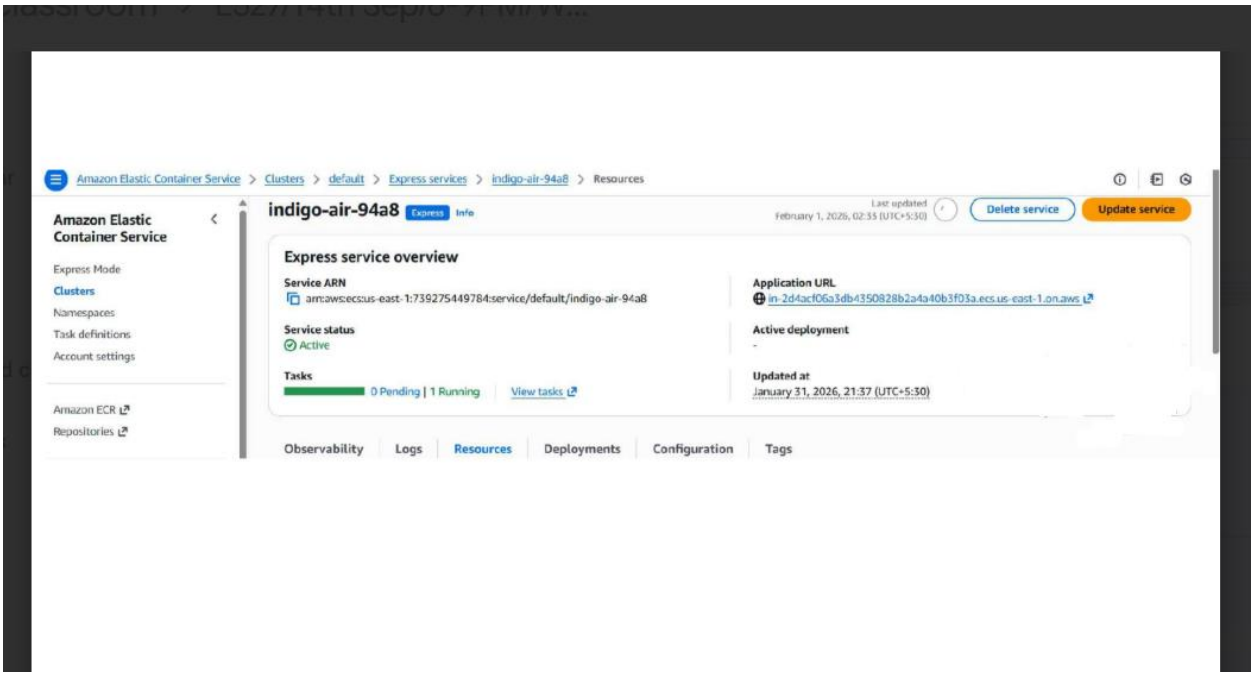


9. Successful Deployment

Screenshot Explanation:

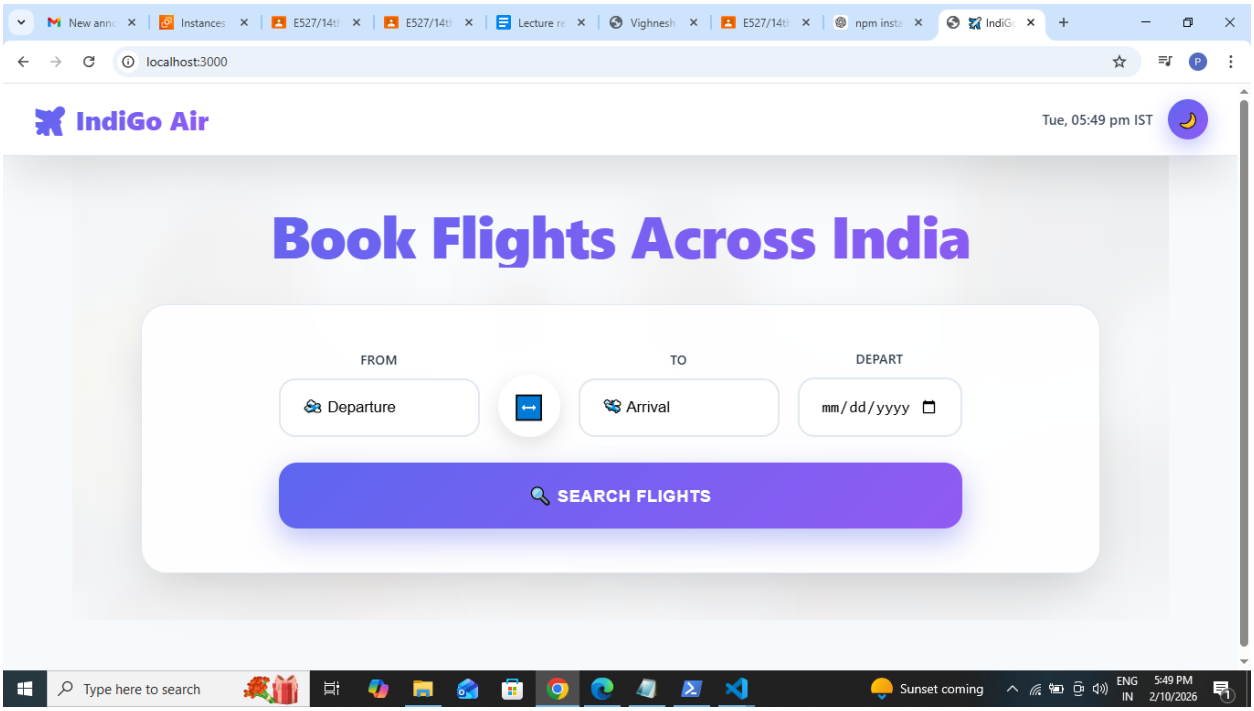
The ECS service reached **RUNNING** state, confirming:

- Image pulled successfully from ECR
- Container started without errors
- Application is live

- 

20. Application Access

Explanation:
The airline ticketing application was accessed using the **ALB DNS name**, verifying end-to-end deployment success.



Conclusion

This project demonstrates:

- Real-world container deployment
- Serverless container management using Fargate
- Secure image storage with ECR